LINGUAGEM DE PROGRAMAÇÃO I

FUNÇÕES
E CONCEITOS EXTRAS

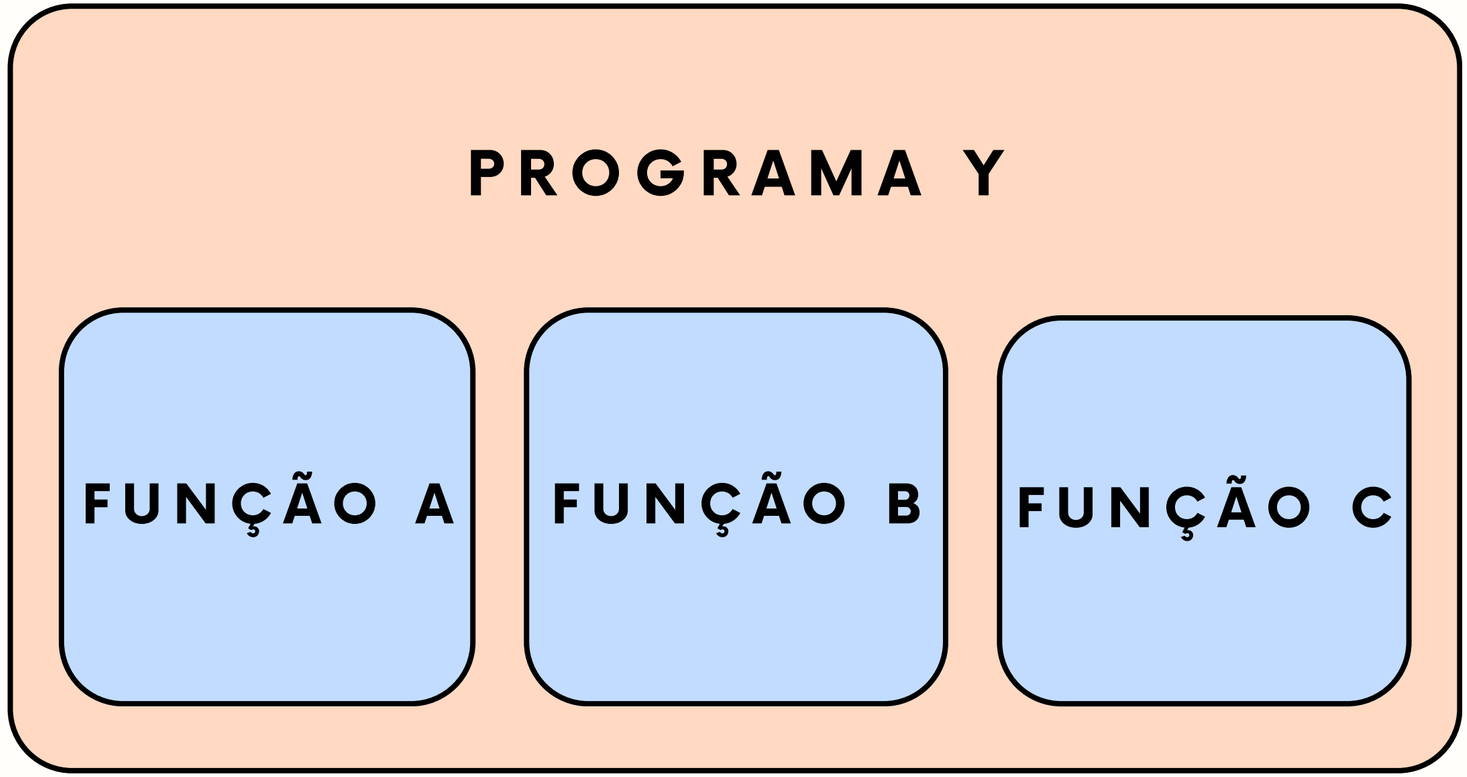
UNIVERSIDADE FEDERAL DO ACRE

PROF.: CLECIO ELIAS
EMAIL.:CLECIO.SILVA@UFAC.BR

Funções em Python

Definição

Funções são blocos de código **organizados** para executar uma determinada tarefa. Elas permitem dividir um programa em partes menores, facilitando sua compreensão, **reutilização**, desenvolvimento e manutenção.

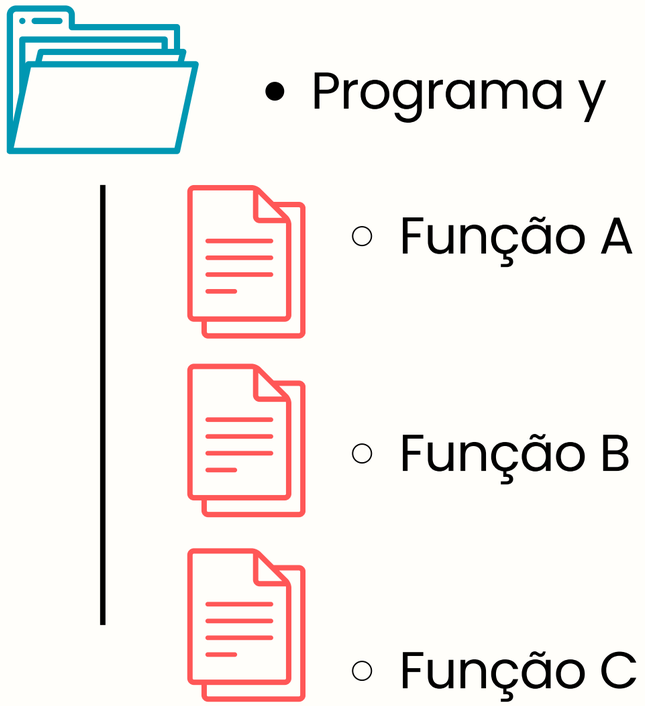


Funções em Python

Por que utilizar funções?

À medida que um programa **cresce**, sua lógica tende a se tornar mais complexa. O uso de funções permite **separar responsabilidades** e desenvolver **partes** do programa de forma independente.

- Dividir problemas grandes em problemas menores;
- Reutilizar código;
- Evitar repetição;
- Facilitar testes;
- Melhorar a leitura do programa;
- Facilitar manutenção e correções.



Funções em Python

Funções já utilizadas em Python

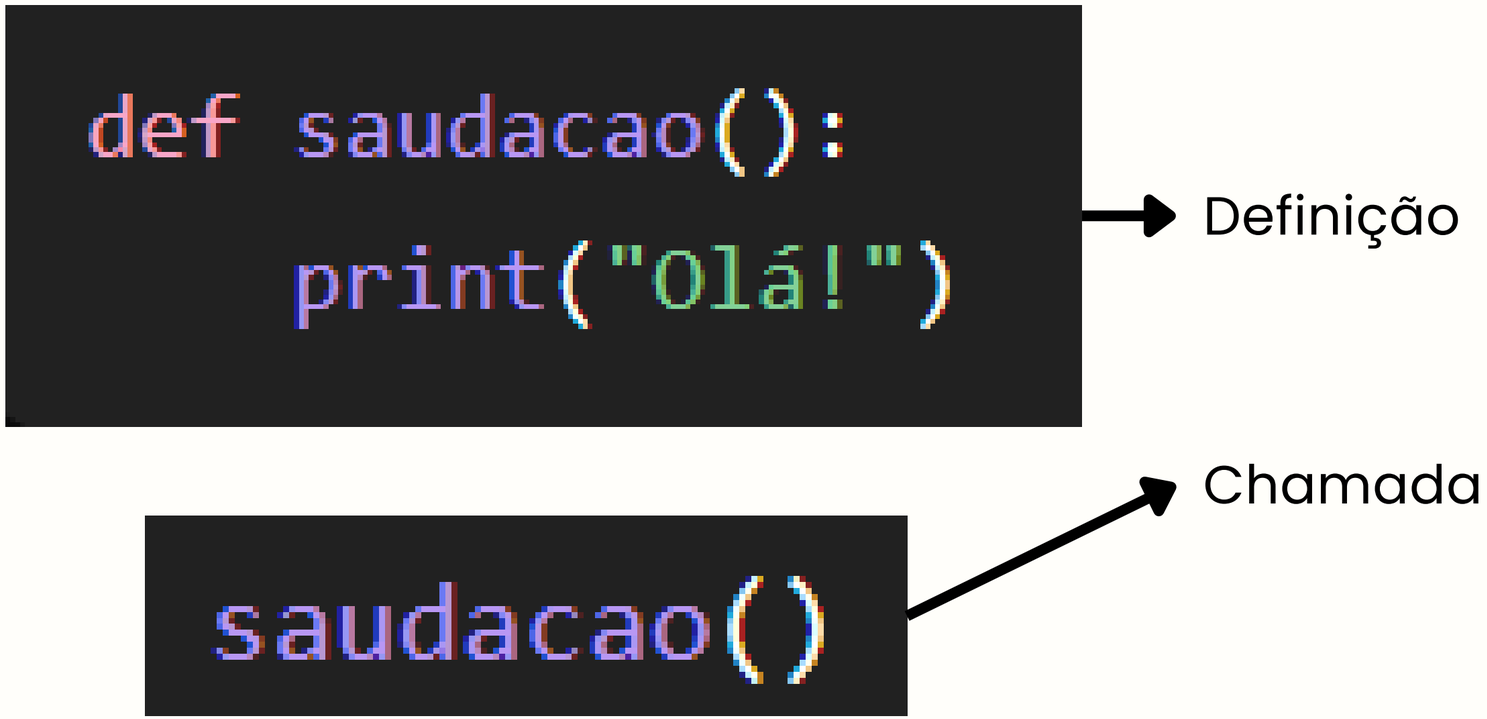
Antes de criar nossas próprias funções, já utilizamos diversas funções disponibilizadas pela linguagem Python. Cada uma recebe determinadas informações, realiza uma tarefa e pode ou não produzir um valor como resultado.

```
print("Olá")
len("Python")
type(10)
int("25")
float("3.5")
input("Digite seu nome: ")
```

Funções em Python

Criando nossas próprias funções

Python permite que o programador defina suas próprias funções. Uma função recebe um nome e pode ser chamada sempre que a tarefa representada por ela precisar ser executada.



Funções em Python

Sintaxe de uma função

Uma função é definida utilizando a palavra reservada **def**, seguida pelo nome da função, seus parâmetros entre parênteses e dois pontos. O corpo da função deve ser escrito de forma indentada.

```
def nome_da_funcao(parametros):
    instruções
    return valor
```

Funções em Python

Funções com e sem retorno

Uma função **pode** produzir um valor utilizando **return**. Quando nenhum valor é explicitamente retornado, a função termina sem fornecer um resultado útil ao programa, correspondendo ao retorno **None**.

```
def dobro(x):
    return x * 2

resultado = dobro(5)
print(resultado)
```

```
def mensagem():
    print("Bem-vindo!")
```

Funções em Python

Definir uma função não significa executá-la

As instruções presentes no corpo de uma função não são executadas no momento de sua definição. A execução ocorre somente quando a função é **chamada**.

```
def ehpar(x):
    return x % 2 == 0
```



```
print(ehpar(13))
```

Funções em Python

Programa principal e subprogramas

Com o uso de funções, um programa pode ser organizado em um **conjunto** de **subprogramas** responsáveis por tarefas específicas e uma parte principal responsável por coordenar sua execução.

```
# Funções

def soma(a, b):
    return a + b

def dobro(x):
    return x * 2
```

```
# Programa principal

n = int(input("Número: "))

resultado = dobro(n)

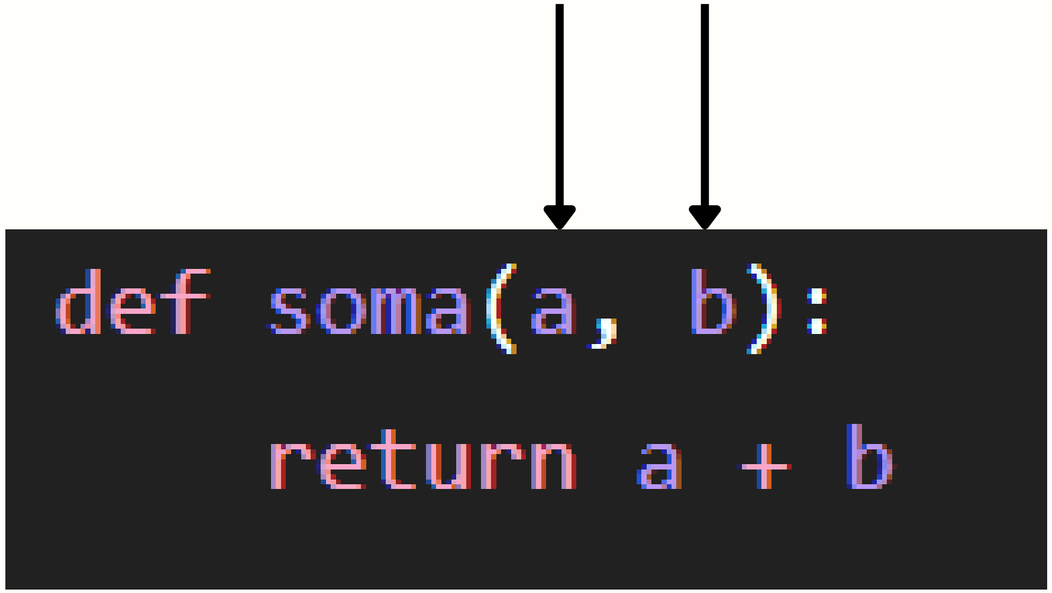
print(resultado)
```

- FUNÇÕES
  - soma()
  - dobro()
- PROGRAMA PRINCIPAL
  - utiliza as funções

Parâmetros

O que são parâmetros?

Parâmetros são variáveis declaradas na definição de uma função para representar os valores que serão utilizados durante sua execução. Eles permitem que uma mesma função seja aplicada a diferentes dados.



Parâmetros

Exemplo: função de multiplicação

Uma função pode utilizar seus parâmetros durante o processamento e produzir um resultado que depende dos valores recebidos em cada chamada.

```
def mult(x, y):
    z = 0

    for i in range(x):
        z = z + y

    return z
```

print(mult(20, 200)) ⟶ mult(20, 200) → 4000

Funções em Python

Funções dentro de expressões

Quando uma função retorna um valor, sua chamada pode participar de atribuições, operações matemáticas e outras expressões do programa.

```
def mult(x, y):
    return x * y
```

```
w = mult(20, 200) + 10

w = mult(a, b) + 10

w = 13 + mult(a + b, mult(c, d))
```

Os argumentos fornecidos a uma função não precisam ser apenas valores constantes. Eles podem ser provenientes de variáveis, expressões ou resultados de outras funções.

Funções em Python

Variáveis locais vs Variáveis globais

Uma variável criada dentro de uma função pertence ao seu escopo local. Ela existe para atender à execução daquela função e não pode ser acessada diretamente por outras partes do programa.

```
def exemplo():
    numero = 10
    print(numero)

exemplo()
```

```
x = 5

def imprimir():
    print(x)

imprimir()
```

Funções em Python

Variáveis locais vs Variáveis globais

Variáveis **locais** e **globais** pertencem a escopos **diferentes**. Assim, uma variável criada dentro de uma função pode possuir o mesmo nome de uma variável externa sem necessariamente representar a mesma variável.

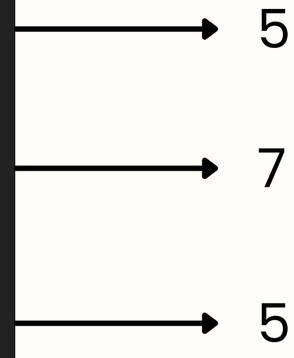
```
x = 5

def muda_e_imprime():
    x = 7
    print("Dentro:", x)

print("Antes:", x)

muda_e_imprime()

print("Depois:", x)
```




```
x = 5

def muda_e_imprime():
    print("Dentro:", x)

print("Antes:", x)

muda_e_imprime()

print("Depois:", x)
```

Funções em Python

Escopo de identificadores

Escopo corresponde à **região do programa** na qual determinado **identificador** pode ser reconhecido e utilizado. Parâmetros e variáveis declaradas dentro de uma função possuem escopo local àquela função.

```
x = 5

def muda_e_imprime():
    x = 7
    print("Dentro:", x)

print("Antes:", x)

muda_e_imprime()

print("Depois:", x)
```

Identificador de escopo global

Identificador de escopo local

Funções em Python

Tipos mutáveis e imutáveis como parâmetros

Ao chamar uma função, seus parâmetros passam a se referir aos objetos fornecidos como argumentos.

Por isso, o comportamento observado pode variar dependendo de o objeto utilizado ser mutável ou imutável.

Imutáveis	**Mutáveis**
int	list
float	dict
str	set
tuple	

Funções em Python

Modificando uma lista dentro de uma função

Quando um objeto **mutável**, como uma lista, é fornecido como argumento, operações realizadas sobre o conteúdo desse objeto dentro da função podem ser observadas posteriormente pelo programa principal.

```
def trocar(valores, pos1, pos2):

    if 0 <= pos1 < len(valores) and 0 <= pos2 < len(valores):

        temp = valores[pos1]
        valores[pos1] = valores[pos2]
        valores[pos2] = temp
```

```
amigos = ["Alice", "Bob", "Bill", "Vitor"]

trocar(amigos, 3, 1)

print(amigos)
```

Conceitos adicionais

Funções: conceitos adicionais

Além da definição convencional de funções com **def**, Python oferece recursos que permitem tornar as funções mais flexíveis, como parâmetros com valores padrão, quantidade variável de argumentos e funções anônimas.

- Valores padrão;
- *args;
- **kwargs;
- Funções lambda;

Conceitos adicionais

Parâmetros com valores padrão

Um parâmetro pode possuir um valor padrão, utilizado automaticamente quando nenhum argumento correspondente é informado na chamada da função.

```
def soma(x=0, y=0):
    print(f"A soma de {x} e {y} é: {x + y}")


soma()
soma(6, 7)
```

Conceitos adicionais

Número variável de argumentos: *args

O parâmetro *args permite que uma função receba uma quantidade variável de argumentos posicionais. Esses argumentos são armazenados em uma tupla.

```python
def soma(*args):
    print(type(args))

    r = 0

    for num in args:
        r += num

    return r

print(soma(10, 20, 30, 40, 50, 60))
```

Conceitos adicionais

Argumentos nomeados: **kwargs

O parâmetro **kwargs permite receber uma quantidade variável de **argumentos nomeados**. Os dados recebidos são organizados em um dicionário, no qual os nomes dos argumentos são utilizados como chaves.

```python
def pessoas_idades(**kwargs):
    print(type(kwargs))

    for chave, valor in kwargs.items():
        print(f"{chave}: {valor}")

pessoas_idades(nome="João", idade=35)
```

Conceitos adicionais

Funções anônimas: lambda

Funções anônimas são **funções simples** definidas por meio da palavra-chave **lambda**. Elas podem ser criadas sem utilizar **def** e são normalmente utilizadas quando uma operação curta precisa ser passada ou executada diretamente.

```
lambda argumentos: expressão
```

```
funcao_soma = lambda x, y: x + y


print(funcao_soma(6, 7))
```

```
print((lambda x, y: x + y)(6, 7))
```

Conceitos adicionais

Parâmetros em funções lambda

Assim como funções convencionais, funções lambda podem receber argumentos **nomeados**, uma quantidade variável de **argumentos posicionais** ou uma quantidade variável de **argumentos nomeados**.

```
# Argumentos nomeados
soma = lambda x, y: x + y
print(soma(x=6, y=7))
```

```
# *args
soma = lambda *args: sum(args)
print(soma(1, 2, 3, 4, 5, 6))
```

```
# **kwargs
soma = lambda **kwargs: sum(kwargs.values())
print(soma(a=1, b=2, c=3))
```

Conceitos adicionais

Condicionais e compreensão de listas em lambda

Uma expressão lambda pode utilizar **expressões condicionais** e **compreensão de listas**, desde que sua definição permaneça composta por uma única expressão.

```
funcao = lambda x, y: x + y if y % 2 == 0 else x * y

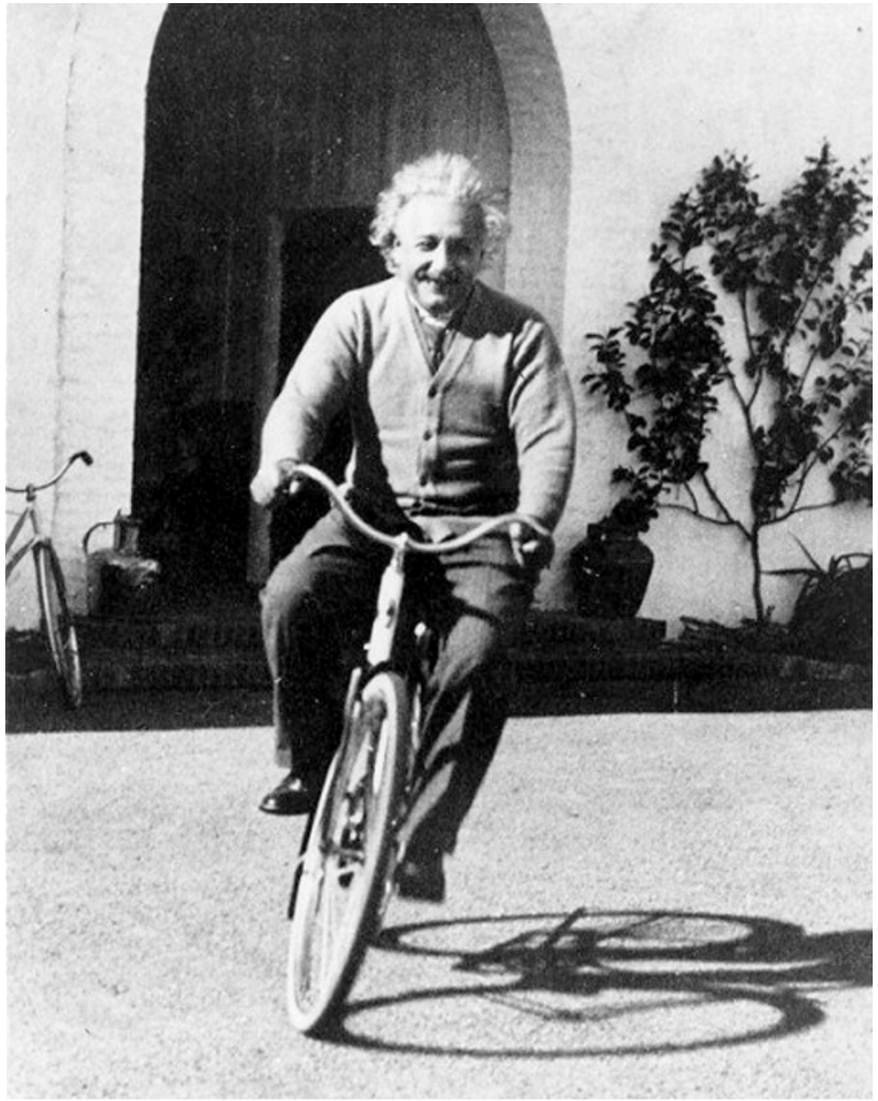
print(funcao(6, 7))
```

```
nomes = ["maria", "pedro", "carlos"]

maiusculas = lambda nomes: [n.upper() for n in nomes]

print(maiusculas(nomes))
```

Exercícios - URI



“A vida é como andar de bicicleta. Para manter o equilíbrio, é preciso se manter em movimento”.

- Einstein

Lista de exercicios!

Conteudos a seguir - programação

Strings + Lista exercicio (extra) Fecha N1	- 12/09 Sábado
Coleções (Tuplas e etc)	- 17/09 Quinta
Funções (+Func anônimas) + Lista exercicio	- 19/09 Sábado
List Comprehension	- 24/09 Quinta
Iteradores e func geradoras + Lista exercicio	- 26/09 Sábado
Exceções	- 08/10 Quinta
Extra (Prova Final caso necessario) + Lista exercicio	- 10/10 Sábado

Próximas listas (Nota completa da N2):

14/09 - 19/09

21/09 - 26/09

27/09 - 10/09